

ASSIGNMENT 5

Instructions for Students

- Create separate .js files for each question.
- Use descriptive variable names, indentation, and comments to explain logic.
- Use only core JavaScript (ES6+) — no frameworks.
- Test using Node.js console or browser console.

Q 1. Write a function **greetUser(name, callback)** that prints "**Hello <name>**" and then executes a callback function **showEndMessage()** which prints "**Welcome to the course!**".

Demonstrate callback flow clearly.

Q 2 . Create a function **applyOperation(numbers, operation)** that takes an array and a callback function.

Use it to:

- Double each number in **[1,2,3,4]**
- Square each number in **[1,2,3,4]**

Q 3 . Create an object **user** with:

- a property **name**
- a method **showName** written as an arrow function
Log the output and explain why **this.name** is **undefined**.
Then fix it by using a normal function.

Q 4 . Create a **constructor function** `Car(brand, model)`. Add a **prototype method** `getDetails()` that prints car brand and model. Create two car objects and call this method to show method sharing via prototype.

Q 5 . Create a **Person** constructor with property **name**. Then create a **Student** constructor that inherits from **Person** and adds property **branch**. Add methods to print both name and branch using prototypes. Demonstrate the **prototype chain** working.

Q 6. Create a prototype chain using constructor functions:

- **Person → Faculty → Professor**
Each should have its own property and method.
Show how a **Professor** object can access all methods up the chain.

Q 7 . Write a function `makeMultiplier(multiplier)` that returns another function to multiply any number by that multiplier.

Example:

```
const triple = makeMultiplier(3);  
console.log(triple(5)); // 15
```

Explain how closure works in this example.

Q 8. Add a custom prototype method `myMap()` to all arrays that behaves like the built-in `map()` function.

Example:

```
[1,2,3].myMap(num => num * 2); // [2,4,6]
```

Q 9 . Rewrite your **Person → Student** inheritance example using **ES6 classes** with `extends` and `super()`.

Show that both class and prototype versions behave the same.